Guía de ejercicios

Arrays en JavaScript

Sesión 1 · Fundamentos de programación

En la guía anterior trabajaste ciclos y funciones con datos sueltos: cada valor vivía en una variable y se perdía cuando llegaba el siguiente. Esta vez vas a guardar muchos valores juntos y volver a ellos cuando quieras. Eso es un array.

La guía tiene cuatro bloques. Los tres primeros se resuelven solo con ciclos, índices y `push`. El cuarto introduce los métodos `forEach`, `map`, `filter` y `find`, y te pide comparar cada uno con lo que ya construiste a mano.

Reglas de la guía

- Solo puedes usar los ciclos `for` y `while`. Nada de `do...while`.
- Los datos se leen con `prompt` y se muestran con `console.log`.
- Hay tres tipos de funciones. Las de **lectura** (`leer...`) pueden usar `prompt`. Las de **mostrar** (`mostrar...`) pueden usar `console.log`. Las de **cálculo** reciben datos por parámetro y retornan un resultado: no usan `prompt` ni `console.log`.
- Nombra las funciones con un verbo en camelCase (`calcularPromedio`, `existeEnLista`) y los arrays en plural (`notas`, `precios`, `invitados`).
- **Bloques 1 a 3:** solo puedes usar `length`, `push` y el acceso por índice. Están prohibidos `includes`, `indexOf`, `reverse`, `sort`, `join`, `Math.max`, `Math.min` y cualquier otro método que haga el trabajo por ti. Sí puedes usar `Math.floor`, `Math.round` y `toFixed`.
- **Bloque 4:** se habilitan `forEach`, `map`, `filter` y `find`.
- Nada de objetos. Todos los arrays de esta guía guardan números o textos.
- Varios ejercicios reutilizan funciones de ejercicios anteriores. Cuando eso pase, cópiala; no la reescribas desde cero.
- **Antes de programar cada ejercicio, haz la prueba de escritorio con los datos del ejemplo.**
- Si el enunciado pide 8 o 10 datos, prueba primero con 3 cambiando la cantidad temporalmente. Cuando funcione, vuelve al valor del enunciado.

En los ejemplos de ejecución, las líneas que empiezan con `[prompt]` muestran lo que el usuario escribe. Las demás son lo que imprime `console.log`.

¿Qué es un array?

Piensa en los casilleros de un gimnasio: una sola fila con espacios numerados donde cada uno guarda una cosa. Un array es eso: una sola variable que guarda varios valores en orden, y a cada posición se llega por su número, llamado **índice**.

```
let precios = [3500, 8000, 2000];
```

Índice	0	1	2
Valor	3500	8000	2000

Tres cosas que tienes que tener claras desde el principio:

- **El primer índice es 0, no 1.** El primer precio es `precios[0]`.
- **`length` dice cuántos elementos hay.** Aquí `precios.length` vale 3, así que el último índice es `precios.length - 1`, es decir 2.
- **Pedir una posición que no existe no da error.** `precios[5]` devuelve `undefined` y el programa sigue como si nada. Por eso los errores con arrays suelen ser silenciosos.

Leer y modificar

```
console.log(precios[1]);   // 8000
precios[1] = 9000;         // cambia el valor de la posición 1
console.log(precios);      // [3500, 9000, 2000]
```

Crear un array vacío y llenarlo

Casi siempre no sabes los valores de antemano: los escribe el usuario. Para eso creas un array vacío y agregas al final con `push`.

```
let nombres = [];
nombres.push("Laura");     // ["Laura"]
nombres.push("Andrés");    // ["Laura", "Andrés"]
console.log(nombres.length);  // 2
```

Recorrer un array

Recorrer es visitar cada posición una vez. El patrón que vas a escribir en casi todos los ejercicios es este:

```
for (let i = 0; i < precios.length; i++) {
  console.log(precios[i]);
}
```

Fíjate en la condición: `i < precios.length`, con `<` y no con `<=`. Si `length` es 3, `i` toma los valores 0, 1 y 2, que son justo los índices válidos.

Arrays como parámetros

Una función puede recibir un array completo y trabajar con él. Así una misma función sirve para 3 datos o para 300.

```
function sumarTodos(numeros) {
  let suma = 0;
  for (let i = 0; i < numeros.length; i++) {
    suma = suma + numeros[i];
  }
  return suma;
}
```

Ojo: cuando le pasas un array a una función, la función trabaja sobre ese mismo array, no sobre una copia. Si dentro haces `numeros[0] = 99`, el array original también cambia. En el bloque 3 esto va a importar.

Prueba de escritorio con arrays

Ya sabes hacer pruebas de escritorio con variables sueltas. Con arrays agregas dos cosas a la tabla: una columna con el **estado completo del array** cuando lo estás llenando, y una columna con el **valor de la posición que estás visitando** (`arreglo[i]`) cuando lo estás recorriendo.

Ejemplo paso a paso

Problema: leer 4 temperaturas, guardarlas en un array y contar cuántas son menores a 20 grados.

```
function contarFrios(temperaturas) {
  let frios = 0;
  for (let i = 0; i < temperaturas.length; i++) {
    if (temperaturas[i] < 20) {
      frios = frios + 1;
    }
  }
  return frios;
}

let temperaturas = [];
for (let i = 0; i < 4; i++) {
  temperaturas.push(Number(prompt("Temperatura " + (i + 1) + ":")));
}
console.log("Días fríos: " + contarFrios(temperaturas));
```

Paso 1: elige los datos y calcula el resultado esperado

Usa datos que puedas resolver mentalmente: 22, 18, 25 y 19. Las menores a 20 son 18 y 19, así que el resultado esperado es **2**. Anótalo antes de empezar.

Paso 2: tabla del llenado

Una fila por cada vuelta del ciclo de lectura. La última columna muestra cómo va quedando el array.

Paso	i	i < 4	Dato leído	temperaturas
1	0	true	22	[22]
2	1	true	18	[22, 18]
3	2	true	25	[22, 18, 25]
4	3	true	19	[22, 18, 25, 19]
5	4	false	—	Sale del ciclo

Paso 3: tabla del recorrido

Ahora entras a `contarFrios`. La columna `temperaturas[i]` te obliga a escribir qué valor estás mirando en cada vuelta.

Paso	i	i < length	temperaturas[i]	< 20	frios
1	0	true	22	false	0
2	1	true	18	true	1

Paso	i	i < length	temperaturas[i]	< 20	frios
3	2	true	25	false	1
4	3	true	19	true	2
5	4	false	—	—	Retorna 2

Paso 4: compara

Esperabas 2 y la tabla da 2. La lógica está bien.

Cómo la prueba detecta un error silencioso

Supón que escribiste `i <= temperaturas.length`. La tabla del recorrido gana una fila:

Paso	i	i <= length	temperaturas[i]	< 20	frios
5	4	true	undefined	false	2

El resultado sigue siendo 2 y el programa no se rompe, así que ejecutándolo nunca lo notarías. Solo lo ves porque la columna `temperaturas[i]` te obliga a escribir `undefined`. En un ejercicio que sume, ese mismo error convierte el resultado en `NaN`.

Regla práctica: si en tu tabla aparece `undefined` en la columna del elemento visitado, tu ciclo da una vuelta de más.

Bloque 1. Crear, llenar y recorrer

Ejercicio 1. Lista del mercado

Qué practica: Array vacío, `push`, recorrido y la diferencia entre el índice y la posición que ve el usuario.

Enunciado:

Pedir cuántos productos se van a comprar, leer el nombre de cada uno y guardarlos en un array. Al final mostrar la lista numerada desde 1 y el total de productos.

Ejemplo de ejecución:

```
[prompt] ¿Cuántos productos? 3
[prompt] Producto 1: arepas
[prompt] Producto 2: queso
[prompt] Producto 3: café
Lista del mercado:
1. arepas
2. queso
3. café
Total: 3 productos
```

Funciones sugeridas:

- `leerProductos(cantidad)`: pide los nombres y retorna el array.
- `mostrarLista(productos)`: imprime la lista numerada y el total.

Pista: Arepas está en el índice 0, pero el usuario debe ver "1.". ¿Qué operación haces al imprimir? ¿Por qué no es buena idea arrancar el ciclo en 1 para resolverlo?

Ejercicio 2. Notas del grupo

Qué practica: Llenar un array con validación, sumar recorriendo y calcular un promedio.

Enunciado:

Leer las notas de 5 aprendices en escala de 0 a 5. Si una nota está fuera del rango, se vuelve a pedir hasta que sea válida. Mostrar todas las notas separadas por comas, el promedio con un decimal y si el grupo aprobó (promedio mayor o igual a 3.0).

Ejemplo de ejecución:

```
[prompt] Nota 1: 4.5
[prompt] Nota 2: 6
Nota inválida, debe estar entre 0 y 5
[prompt] Nota 2: 3
[prompt] Nota 3: 2.5
[prompt] Nota 4: 4
[prompt] Nota 5: 3.5
Notas: 4.5, 3, 2.5, 4, 3.5
Promedio: 3.5
El grupo aprobó
```

Funciones sugeridas:

- `leerNotaValida(numero)`: pide la nota número `numero` hasta que sea válida y la retorna.

- `calcularPromedio(numeros)`: retorna el promedio de cualquier array de números.
- `unirConComas(lista)`: retorna un texto con los elementos separados por coma y espacio, sin coma al final. La vas a reutilizar en casi toda la guía.

Pista: Si mañana son 30 aprendices en vez de 5, ¿cuántas líneas de `calcularPromedio` tienes que cambiar? Si la respuesta no es "ninguna", revisa qué usaste como límite del ciclo. Para `unirConComas`: ¿cómo sabe el ciclo que está en el último elemento?

Ejercicio 3. Primero, último y del medio

Qué practica: `length`, último índice y cálculo de índices.

Enunciado:

Leer N números (N lo da el usuario y debe ser al menos 1). Mostrar el primero, el último y el del medio. Si N es par hay dos del medio: mostrar ambos.

Ejemplo de ejecución:

```
[prompt] ¿Cuántos números? 5
[prompt] Número 1: 8
[prompt] Número 2: 3
[prompt] Número 3: 9
[prompt] Número 4: 1
[prompt] Número 5: 7
Primero: 8
Último: 7
Del medio: 9

(Con N = 4 y los datos 10, 20, 30, 40, la última línea sería: Del medio: 20 y 30)
```

Funciones sugeridas:

- `leerNumeros(cantidad)`: retorna el array con los números leídos.
- `obtenerUltimo(numeros)`: retorna el último elemento sin importar el tamaño del array.
- `mostrarMedio(numeros)`: imprime el elemento o los dos elementos del medio.

Pista: Antes de programar, calcula a mano con N = 1, 2, 5 y 6: ¿qué da `length / 2` en cada caso y qué índice necesitas realmente? ¿Qué pasa si N es 1?

Ejercicio 4. La semana de ventas

Qué practica: Arrays paralelos: dos arrays distintos que usan el mismo índice para hablar del mismo dato.

Enunciado:

Un local de empanadas registra sus ventas diarias. El programa arranca con este array fijo:

```
let dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado",
"Domingo"];
```

Pedir la venta de cada día usando el nombre del día en el mensaje, guardarlas en otro array y mostrar un reporte día por día con el total de la semana.

Ejemplo de ejecución:

```
[prompt] Venta del Lunes: 120000
[prompt] Venta del Martes: 95000
[prompt] Venta del Miércoles: 110000
[prompt] Venta del Jueves: 130000
[prompt] Venta del Viernes: 210000
[prompt] Venta del Sábado: 260000
[prompt] Venta del Domingo: 180000
Lunes: $120000
Martes: $95000
Miércoles: $110000
Jueves: $130000
Viernes: $210000
Sábado: $260000
Domingo: $180000
Total semana: $1105000
```

Funciones sugeridas:

- `leerVentas(dias)`: pide una venta por cada día y retorna el array de ventas.
- `calcularTotal(numeros)`: retorna la suma de un array de números.
- `mostrarReporte(dias, ventas)`: imprime el reporte.

Pista: Si alguien agrega "Festivo" al array `dias`, ¿tu programa pide una venta más sin que cambies nada? ¿Qué tendría que pasar para que `dias[3]` y `ventas[3]` dejen de hablar del mismo día?

Bloque 2. Buscar y contar

Ejercicio 5. El día de mejor venta

Qué practica: Buscar el mayor y el menor de un array, y su posición, sin `Math.max` ni `Math.min`.

Enunciado:

Copia la lectura del ejercicio 4. Mostrar el día con mayor venta, el día con menor venta y la diferencia entre ambos.

Ejemplo de ejecución:

```
(Con los mismos datos del ejercicio 4)
Mejor día: Sábado ($260000)
Peor día: Martes ($95000)
Diferencia: $165000
```

Funciones sugeridas:

- `buscarPosicionMayor(numeros)`: retorna el índice del mayor.
- `buscarPosicionMenor(numeros)`: retorna el índice del menor.

Pista: ¿Por qué es más útil que la función retorne la posición y no el valor? ¿Con qué inicializas "el mayor hasta ahora": con 0 o con el primer elemento? Haz la prueba de escritorio con ventas negativas (devoluciones) para decidir.

Ejercicio 6. Lista de invitados

Qué practica: Búsqueda lineal que retorna un booleano, `while` con valor centinela.

Enunciado:

El programa tiene la lista fija de invitados:

```
let invitados = ["ana", "carlos", "luisa", "pedro", "sofía"];
```

El portero escribe nombres y para cada uno el programa dice si puede entrar. Termina cuando escribe "fin". Al final muestra cuántos entraron y cuántos fueron rechazados. Prohibido `includes` e `indexOf`.

Ejemplo de ejecución:

```
[prompt] Nombre: luisa
luisa puede entrar
[prompt] Nombre: jorge
jorge no está en la lista
[prompt] Nombre: ana
ana puede entrar
[prompt] Nombre: fin
Entraron: 2
Rechazados: 1
```

Funciones sugeridas:

- `existeEnLista(lista, valor)`: retorna `true` si `valor` está en `lista` y `false` si no.

Pista: Si el nombre está en la posición 0, ¿tu función sigue revisando las otras 4 posiciones? ¿Cómo haces que se detenga apenas lo encuentre? Reto opcional: que "Ana" con mayúscula también entre.

Ejercicio 7. Calificaciones de domicilios

Qué practica: Array de contadores, donde el índice representa una categoría.

Enunciado:

Una app de domicilios pide a los clientes calificar de 1 a 5 estrellas. Leer 10 calificaciones validando el rango. Contar cuántas hubo de cada valor usando **un array de 5 contadores** (no cinco variables) y mostrar un gráfico con asteriscos. Mostrar también la calificación más frecuente.

Ejemplo de ejecución:

```
[prompt] Calificación 1: 5
[prompt] Calificación 2: 4
(…se piden las 10. Datos completos: 5, 4, 5, 3, 5, 4, 2, 5, 4, 5)
Estrellas 1:  (0)
Estrellas 2: * (1)
Estrellas 3: * (1)
Estrellas 4: *** (3)
Estrellas 5: ***** (5)
Más frecuente: 5 estrellas
```

Funciones sugeridas:

- `contarPorEstrellas(calificaciones)`: retorna un array de 5 contadores.
- `repetirCaracter(caracter, veces)`: retorna un texto con el carácter repetido.
- `buscarPosicionMayor(numeros)`: la del ejercicio 5.

Pista: Si llega una calificación de 4, ¿en qué índice del array de contadores la sumas? Escribe la relación entre estrellas e índice antes de programar. ¿Con qué valores tiene que arrancar el array de contadores?

Ejercicio 8. Ventas sobre el promedio

Qué practica: Dos recorridos donde el segundo depende del resultado del primero.

Enunciado:

Con la lectura del ejercicio 4, calcular el promedio diario redondeado a pesos, mostrar qué días estuvieron por encima del promedio y cuántos fueron.

Ejemplo de ejecución:

```
(Con los mismos datos del ejercicio 4)
Promedio diario: $157857
Días por encima del promedio:
Viernes: $210000
Sábado: $260000
Domingo: $180000
Total: 3 días
```

Funciones sugeridas:

- `calcularPromedio(numeros)`: la del ejercicio 2.
- `contarMayoresQue(numeros, limite)`: retorna cuántos elementos superan `limite`.
- `mostrarDiasSobre(dias, ventas, limite)`: imprime los días que superan `limite`.

Pista: ¿Puedes saber si el lunes está por encima del promedio en el mismo ciclo en el que estás sumando las ventas? ¿Por qué no?

Bloque 3. Construir arrays nuevos

En este bloque cada función de cálculo retorna un array **nuevo**. El array que recibe por parámetro no se toca.

Ejercicio 9. Al revés sin reverse

Qué practica: Recorrer de atrás hacia adelante y construir un array nuevo sin modificar el original.

Enunciado:

Leer N palabras. Crear una función que retorne un array nuevo con las palabras en orden inverso, sin modificar el original y sin usar `reverse`. Mostrar ambos arrays para demostrar que el original quedó igual.

Ejemplo de ejecución:

```
[prompt] ¿Cuántas palabras? 4
[prompt] Palabra 1: sol
[prompt] Palabra 2: luna
[prompt] Palabra 3: mar
[prompt] Palabra 4: río
Original: sol, luna, mar, río
Invertido: río, mar, luna, sol
```

Funciones sugeridas:

- `invertir(lista)`: retorna el array nuevo invertido.
- `unirConComas(lista)`: la del ejercicio 2.

Pista: ¿Cuál es el índice del primer elemento que debes leer? ¿Qué condición evita que el ciclo se salte la posición 0 o intente leer la posición -1?

Ejercicio 10. Pares e impares

Qué practica: Repartir los elementos de un array en dos arrays nuevos según una condición.

Enunciado:

Leer 8 números enteros. Construir dos arrays nuevos, uno con los pares y otro con los impares, en el orden en que llegaron. Mostrar ambos con su cantidad. Si alguno queda vacío, mostrar "(ninguno)".

Ejemplo de ejecución:

```
[prompt] Número 1: 12
(…se piden los 8. Datos completos: 12, 7, 4, 9, 20, 15, 3, 8)
Pares (4): 12, 4, 20, 8
Impares (4): 7, 9, 15, 3
```

Funciones sugeridas:

- `esPar(numero)`: la de la guía de ciclos.
- `filtrarPares(numeros)` y `filtrarImpares(numeros)`: retornan los arrays nuevos.

Pista: ¿Qué retorna tu `unirConComas` si le pasas un array vacío? Prueba con 8 números pares antes de entregar. Y si te dan -3, ¿tu programa lo reconoce como impar? Haz la cuenta de `-3 % 2`.

Ejercicio 11. Sin repetidos

Qué practica: Construir un array consultándolo mientras se construye, y reutilizar funciones.

Enunciado:

Leer 10 números. Construir un array nuevo sin repetidos que conserve el orden en que apareció cada número por primera vez. Mostrar cuántos repetidos se eliminaron. Prohibido `includes`, `indexOf` y `Set`.

Ejemplo de ejecución:

```
[prompt] Número 1: 3
(…se piden los 10. Datos completos: 3, 7, 3, 1, 7, 7, 9, 1, 2, 3)
Sin repetidos: 3, 7, 1, 9, 2
Se eliminaron 5 repetidos
```

Funciones sugeridas:

- `eliminarRepetidos(numeros)`: retorna el array nuevo.
- `existeEnLista(lista, valor)`: la del ejercicio 6.

Pista: Cuando llega el segundo 7, ¿en cuál de los dos arrays lo buscas: en el original o en el que estás construyendo? Haz la prueba de escritorio con una columna para cada array.

Ejercicio 12. Rotar la fila

Qué practica: Índices calculados con fórmula y el operador `%`.

Enunciado:

En una fila de turnos, rotar k posiciones a la derecha significa que los últimos k pasan al inicio. Leer N nombres y el valor de k, y retornar un array nuevo rotado. k puede ser mayor que N: rotar 7 en una fila de 5 es lo mismo que rotar 2.

Ejemplo de ejecución:

```
[prompt] ¿Cuántas personas? 5
[prompt] Persona 1: Ana
[prompt] Persona 2: Beto
[prompt] Persona 3: Caro
[prompt] Persona 4: Dani
[prompt] Persona 5: Eli
[prompt] ¿Cuántas posiciones rotar? 2
Fila original: Ana, Beto, Caro, Dani, Eli
Fila rotada: Dani, Eli, Ana, Beto, Caro

(Con k = 7 el resultado debe ser el mismo)
```

Funciones sugeridas:

- `rotarDerecha(lista, k)`: retorna el array nuevo rotado.

Pista: El elemento que está en la posición i, ¿a qué posición llega después de rotar? Escribe la fórmula y pruébala a mano con i = 4 y k = 2 en una fila de 5. ¿Qué operador evita que el resultado se salga del array?

Bloque 4. Métodos de array

Todo lo que hiciste en los bloques anteriores con `for` se puede escribir más corto con métodos que ya trae JavaScript. Ahora que sabes lo que hacen por dentro, puedes usarlos sin que sean magia.

Funciones que reciben funciones

Estos métodos reciben una función como parámetro, llamada **callback**. El método recorre el array y llama tu función una vez por cada elemento. Lo más común es escribirla como función flecha:

```
let precios = [3500, 8000, 2000];

// forEach: ejecuta algo por cada elemento. No retorna nada.
precios.forEach((precio, i) => {
  console.log(i + ": " + precio);
});

// map: retorna un array nuevo del mismo tamaño con cada elemento transformado.
let dobles = precios.map((precio) => precio * 2);      // [7000, 16000, 4000]

// filter: retorna un array nuevo solo con los que cumplen la condición.
let baratos = precios.filter((precio) => precio < 5000); // [3500, 2000]

// find: retorna el primer elemento que cumple, o undefined si ninguno cumple.
let caro = precios.find((precio) => precio > 5000);      // 8000
```

Si la función flecha tiene una sola expresión y no lleva llaves, retorna ese valor sola. Si lleva llaves, tienes que escribir `return`.

Método	Qué retorna	Tamaño del resultado	Lo que ya hiciste a mano
`forEach`	Nada (`undefined`)	—	Recorrer con `for` (ejercicio 1)
`map`	Array nuevo	Igual al original	`invertir` y `rotarDerecha` también crean arrays nuevos
`filter`	Array nuevo	Igual o menor	`filtrarPares` (ejercicio 10)
`find`	Un elemento o `undefined`	—	`existeEnLista` (ejercicio 6), pero retorna el elemento

Ejercicio 13. La lista con forEach

Qué practica: `forEach` usando el índice del callback.

Enunciado:

Reescribe `mostrarLista` del ejercicio 1 usando `forEach` en lugar de `for`. La salida debe ser idéntica. Al final del archivo, responde en un comentario: ¿se puede detener un `forEach` a la mitad, como hiciste en `existeEnLista`? ¿Qué te dice eso sobre cuándo usarlo y cuándo no?

Ejemplo de ejecución:

```
(La misma salida del ejercicio 1)
```

Funciones sugeridas:

- `mostrarLista(productos)`: versión con `forEach`.

Pista: El callback de `forEach` puede recibir más de un parámetro. ¿Cuál necesitas para imprimir "1.", "2.", "3."?

Ejercicio 14. Precios con IVA

Qué practica: `map` para transformar cada elemento, y comparar con la versión manual.

Enunciado:

Leer los precios de N productos sin IVA. Generar con `map` un array nuevo con el precio más IVA del 19%, redondeado a pesos con `Math.round`. Mostrar ambos arrays y el total con IVA. Escribe también la versión con `for` en otra función y comprueba que las dos den el mismo resultado.

Ejemplo de ejecución:

```
[prompt] ¿Cuántos productos? 3
[prompt] Precio 1: 10000
[prompt] Precio 2: 25000
[prompt] Precio 3: 4200
Sin IVA: 10000, 25000, 4200
Con IVA: 11900, 29750, 4998
Total con IVA: $46648
```

Funciones sugeridas:

- `calcularConIva(precio)`: retorna el precio de un solo producto con IVA.
- `aplicarIvaConMap(precios)` y `aplicarIvaConFor(precios)`: retornan el array nuevo.
- `calcularTotal(numeros)`: la del ejercicio 4.

Pista: ¿`map` modifica el array `precios`? Imprímelo después de llamar la función y compruébalo. ¿Qué pasa si escribes la función flecha con llaves y se te olvida el `return`?

Ejercicio 15. Cobertura de domicilios

Qué practica: `filter` y `find`, y combinar un método con `length`.

Enunciado:

Leer las distancias en kilómetros de N pedidos del día. El envío es gratis hasta 3 km y la cobertura máxima es de 10 km. Con `filter`, obtener los pedidos con envío gratis. Con `find`, el primer pedido fuera de cobertura. Mostrar también cuántos pedidos pagan envío (más de 3 km y hasta 10 km). Si ningún pedido está fuera de cobertura, mostrar "Todos los pedidos están en cobertura".

Ejemplo de ejecución:

```
[prompt] ¿Cuántos pedidos? 6
[prompt] Distancia 1: 2.5
[prompt] Distancia 2: 7
[prompt] Distancia 3: 1.2
[prompt] Distancia 4: 12
```

```
[prompt] Distancia 5: 3
[prompt] Distancia 6: 15
Envío gratis (3): 2.5, 1.2, 3
Pagan envío: 1
Primer pedido fuera de cobertura: 12 km
```

Funciones sugeridas:

- `obtenerEnvioGratis(distancias)`: con `filter`.
- `contarConEnvio(distancias)`: con `filter` y `length`.
- `buscarPrimeroFuera(distancias)`: con `find`.

Pista: Según el enunciado, ¿un pedido de 3 km exactos tiene envío gratis? Revisa si usaste `<` o `<=`. Y si `find` no encuentra nada, ¿qué valor llega a tu `console.log` y cómo lo detectas antes de imprimir "undefined km"?

Antes de entregar

- Cada ejercicio tiene su prueba de escritorio hecha con los datos del ejemplo, y el resultado de la tabla coincide con lo que imprime tu programa.
- Ninguna función de cálculo usa `prompt` ni `console.log`.
- En los bloques 1 a 3 no usaste ningún método prohibido.
- Probaste al menos un caso borde por ejercicio: un solo dato, un array vacío, valores repetidos o valores en el límite.
- Las funciones que se reutilizan son las mismas en todos los ejercicios. Si encontraste un error en una, lo corregiste en todas partes.